Send Values to Generators

`value = yield x` turns a generator into a coroutine: `send()` pushes data in while `yield` hands data out.

yield is an expression

```
def echo():
    while True:
        # pause and wait for a value
        received = yield
        print("got", received)

e = echo()
# "prime": run to first yield
next(e)
e.send("hello")
e.send(42)
```

OUTPUT

```
got hello
got 42
```

Two-way traffic

```
def running_average():
    total, count, avg = 0, 0, None
    while True:
        # hand avg out, take x in
        x = yield avg
        total += x
        count += 1
        avg = total / count

ra = running_average()
next(ra)                        # prime
print(ra.send(10))
print(ra.send(20))
print(ra.send(30))
```

OUTPUT

```
10.0
15.0
20.0
```

What send(v) does

- Resumes the generator; the paused `yield` **evaluates to `v`**.
- Runs until the **next** `yield`.
- The value after that `yield` is what `send()` returns.
- `next(g)` is exactly `g.send(None)`.

First call must be None (priming)

```
g = echo()
try:
    g.send("too early")
except TypeError as e:
    print(e)
```

OUTPUT

```
can't send non-None value to a just-started generator
```

throw() and close()

```
def worker():
    try:
        while True:
            x = yield
            print("work on", x)
    except ValueError:
        print("caught ValueError")
        yield "recovered"
    finally:
        print("cleanup")

w = worker()
next(w)
w.send("job1")
print(w.throw(ValueError))
w.close()
```

OUTPUT

```
work on job1
caught ValueError
recovered
cleanup
```

Call	Effect
`next(g)`	resume, sending `None`
`g.send(v)`	resume; `yield` evaluates to `v`
`g.throw(Exc)`	raise `Exc` at the paused `yield`
`g.close()`	raise `GeneratorExit`; generator ends

Auto-prime decorator

```
def primed(fn):
    def wrapper(*a, **kw):
        g = fn(*a, **kw)
        next(g)              # advance to first yield
        return g
    return wrapper

@primed
def acc():
    total = 0
    while True:
        total += yield total

a = acc()
print(a.send(5), a.send(10))
```

OUTPUT

```
5 15
```